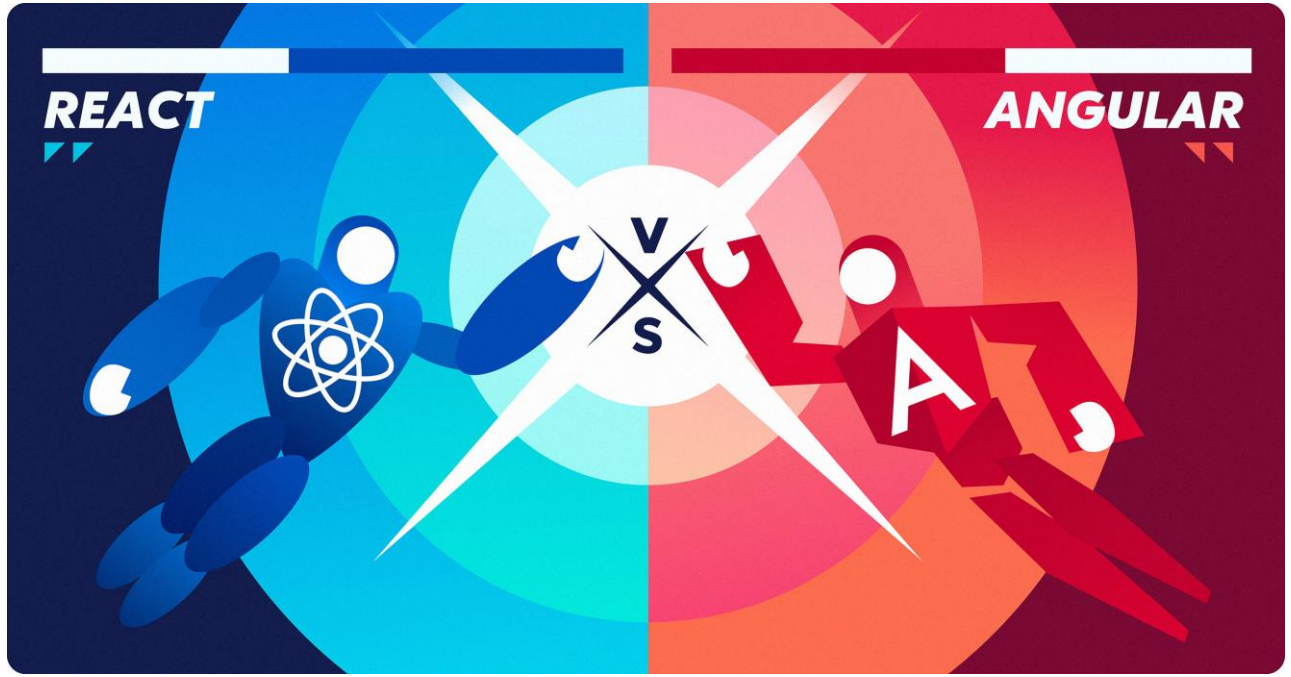


TECHNOLOGIES WEB II (React & Angular)

Professeur	Dr. Fadwa LACHHAB
Niveau	4eme année
Année	2024-2025

Partie 2 : Angular

Semaine 1
- Presentation de Angular
- Environnement de développement
- Components

Semaine 2
- Data Binding/Directives
- Observables/ Forms

Semaine 3
- Services / Routing

Semaine 4
- Interaction avec Base de Données : FireBase

Semaine 5
- Pipes /HTTP
- Authentification

Semaine 6
- Examen TP

Angular

Data Binding

- Introduction
- Types du **DataBinding** :
  - Du component vers la template
  - De la template vers le component
  - Les deux sens (**NgModel**)

Data Binding : Introduction

- Le **databinding** est l'une des fonctionnalités les plus impressionnantes dans Angular.
- Il s'agit d'un mécanisme qui permet la synchronisation instantanée entre la vue (HTML Template) et les données de l'application.
- Il permet de faire la communication entre le component et sa template.
- Angular propose plusieurs types du databinding qui seront utilisés selon le besoin.

Data Binding : Types

- Il y a trois sens pour le flux:

1. Du component vers la template
2. De la template vers le component
3. Dans le deux sens (si on change dans le html donc la valeur du component change et inversement si suite à un traitement, la valeur du component ou sa propriété change donc le navigateur et l'affichage se rafraîchit automatiquement et instantanément).

Data Binding : Types

Classe du composant

```
export class TestComponent {
  counter : number =1;
  name: string="";
  increment() {
    ++this.counter;
  }

  decrement() {
    --this.counter;
  }
}
```

Output Data

String interpolation : {{data}}
Property Binding: [property]="data"

React to User Events

Event Binding : (clik)="doSome()"

Two Way Bindings

Two way binding: [(ngModel)]="data"

Partie HTML du composant

Mohamed

Name is : Mohamed

- Counter: 1 +

```
<input type="text" [(ngModel)]="name">
<label>Name is : {{name}}</label>
<button (click)="decrement()" [disabled]="counter==0">
-
</button>
<label>Counter: {{counter}}</label>
<button (click)="increment()" [disabled]="counter==10">
+
</button>
```

Data Binding : Types

Uni-Directionnel	Du modèle -> Template	1.Interpolation: {{ NomProp }} 2.Property Binding: [property] = “data”
Uni-Directionnel	Du Template -> Modèle (Exemple: l'événement TextChanged)	3.Event Binding * (click) = "....." * on-click
Bi-Directionnel	S’il change dans l’un, la modification se répercute instantanément	4.Two-WAy-DataBinding input [(ngModel)]= « nomVariable" >

Interpolation (Sens: du component vers l'affichage)

- C'est un type de databinding du Modèle vers la template.
- Il suffit, tout simplement, de mettre le nom de la propriété du component dans double accolades ouvrantes et fermante comme suivant **{{ }}**.

src > app > TS app.component.ts > AppComponent > cours

```
1   import { Component } from '@angular/core';
2
3   @Component({
4     selector: 'app-root',
5     templateUrl: './app.component.html',
6     styleUrls: ['./app.component.css']
7   })
8   export class AppComponent  {
9     title1: string = 'Bonjour Angular!';
10    description: string = 'Ceci est un exemple d\'interpolation.';
```

```
<div>
  <h1>{{ title1 }}</h1>
  <p>{{ description }}</p>
</div>
```

Bonjour Angular!

Ceci est un exemple d'interpolation.

UNIVERSIAPOLIS Université Internationale d'Agadir الجامعة الدولية لأكادير
ECOLE POLYTECHNIQUE D'AGADIR

Binding des événements (Sens: de l'affichage vers le component)

Dans le cas précédent (Interpolation et PropertyBinding), l'envoi des données est unidirectionnel et c'est du component vers la template. Maintenant on va étudier un autre binding unidirectionnel mais dans le sens inverse.

C.à.d. comment envoyer des données à partir de la template vers le component.

Angular permet de reagir aux évènements javascript déclenchés depuis la vue.

src/app.component.html

```
<button
        type="button"
        (click)="buy()">BUY</button>
```

src/app.component.ts

```
...
export class AppComponent {

    buy() {
        ...
    }

}
```

Binding des événements (Sens: de l'affichage vers le component)

Exemple :

```
export class AppComponent  {
  title1: string = 'Bonjour Angular!';
  description: string = 'Ceci est un exemple d\'interpolation.';
  msg: string = 'Bienvenue sur Angular!';
  // Méthode pour changer le message
  changeMessage() {
    this.msg = 'Le message a été modifié!';
  }
```

```
<div>
  <h1>{{ title1 }}</h1>
  <p>{{ description }}</p>
  <h1>{{ msg }}</h1>

<!-- Lien l'événement click à la méthode changeMessage -->
<button (click)="changeMessage()">Changer le message</button>
</div>
```

Bienvenue sur Angular!

Changer le message

Le message a été modifié!

Changer le message

Binding Bidirectionnel en utilisant NgModel (Dans les deux sens))

- Le Binding Bidirectionnel permet de faire une synchronisation instantanée entre la template et la valeur de la propriété du component. Cette synchronisation est dans les deux sens.
- **NgModel** est une directive très utilisée dans Angular.
- Important: Pour que le binding bidirectionnel fonctionne, il faut obligatoirement importer FormsModule dans le fichier Module associé.

Importer FormsModule pour ngModel

app.module.ts

```
import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core';

import { AppComponent } from './app.component';
import { AboutComponent } from './about/about.component';


import { ContactsComponent } from
'./contacts/contacts.component';

import {FormsModule} from "@angular/forms";

@NgModule({
  declarations: [
    AppComponent,
    AboutComponent,
    ContactsComponent
  ],
  imports: [
    BrowserModule, FormsModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Binding Bidirectionnel en utilisant NgModel (Dans les deux sens))

- Exemple :

```
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent  {

  twoways: string = '';
```

```
<h1>Bienvenue dans le Binding Bidirectionnel</h1>

<!-- Input lié au modèle 'name' avec ngModel -->
<input [(ngModel)]="twoways" placeholder="Entrez votre nom">

<p>Votre nom est: {{ twoways }}</p>
```

Bienvenue dans le Binding Bidirectionnel

lachhab

Votre nom est: lachhab

Décorateurs Dans Angular

Décorateurs

- Elle permet d'assurer les canaux de communication entre les différents components, dans les deux sens (du fils vers parent et vis versa).
- La question qui se pose: Comment deux components peuvent se communiquer ? comment, le deuxième component peut répondre à la demande du premier ? le deuxième component, comment peut-il envoyer des objets en tant que réponse à la demande.

➔ On va utiliser les décorateurs dans le cadre de la communication inter-components et pour d'autres besoins.

Décorateurs

- D'une manière simplifiée "**Decorators**" sont des fonctions préfixées par le symbole "**@**".
- Les Decorators permettent d'ajouter une condition ou une information sure (classe, propriété, méthode, ou paramètre) auquel il est appliqué.
- On a déjà utilisé le décorateur @component qui permet d'ajouter des informations sur le component comme le '**selector**', **templateUrl**, **styleUrls** ….

@Input decorator

Partage des données entre Parent et Child Components

- On utilise **@input** pour dire que cette propriété est une entrée (input) pour ce component.
- On déclare une propriété. Puis on ajoute le décorateur **@input** pour dire que notre component peut recevoir des messages à travers cette propriété.

➔un component A, possède une propriété décorée par un input, alors un component B peut communiquer avec A en modifiant la valeur de la propriété.

- **@input**, peut décorer n'importe quel type de propriété (**number**, **string**, **array** ou même un objet défini par le développeur).

@Input decorator

Partage des données entre Parent et Child Components

TS child.component.ts

src > app > child > TS child.component.ts > ChildComponent

```
1   import { Component , Input } from '@angular/core';
2
3   @Component({
4     selector: 'app-child',
5     templateUrl: './child.component.html',
6     styleUrls: ['./child.component.css']
7   })
8   export class ChildComponent {
9     @Input() commentaire: string = '';
10  }
```

child.component.html

src > app > child > child.component.html > div

Go to component

```
1  <div>
2      <h3>Message du parent:</h3>
3      <!-- Affichage de la valeur transmise par le parent -->
4      <p>{{ commentaire }}</p>
5  </div>
```


UNIVERSIAPOLIS Université Internationale d'Agadir الجامعة الدولية لأكادير
ECOLE POLYTECHNIQUE D'AGADIR

@Input decorator

TS app.component.ts

src > app > TS app.component.ts > AppComponent > isFirstName

```
// Propriété que le parent va envoyer au composant enfant
parentCommentaire: string = 'Bonjour, je suis le composant parent!';
```

app.component.html 3

src > app > app.component.html > div

```
<div>
  <h1>Composant Parent</h1>
  <app-child [commentaire]="parentCommentaire"></app-child>
  <!-- Binding de la propriété 'parentMessage' au composant enfant -->
</div>
```

localhost:4200

screen shot 2018-0... Google Boîte de réception -...

Composant Parent

Message du parent:

Bonjour, je suis le composant parent!

@Output decorator

- le décorateur **@Output** est très similaire à l'input sauf que l'output est utilisé dans l'autre sens de la communication. c.à.d. que le deuxième component peut répondre le premier component.
- Un point particulier avec ce décorateur: Il est toujours **bindé** à une propriété de type **"EventEmitter"**.
- **Exemple**: la propriété suivante "**Prop**" est visible chez le component émetteur avec le nom suivant "**MaProp**". Notez bien que cette propriété utilise EventEmitter pour retourner un string.

```
@Output('MaProp')
Prop = new EventEmitter<string>();
```

@Output decorator

app.component.html 3 ● | app.component.ts

src > app > app.component.html > div > div

```
16  <div>
17    <h1>Composant Parent</h1>
18      <!-- Appel du composant enfant -->
19      <app-child
20      [commentaire]="parentCommentaire"
21      (messageAuParent)="recevoirMessage($event)">
22    </app-child>
23    <p><b>Message reçu de l'enfant :</b> {{ enfantMessage }}</p>
24  </div>
```

app.component.ts

src > app > app.component.ts > AppCompor

```
21  //recevoir message
22  enfantMessage: string = '';
23
24  recevoirMessage(msg: string) {
25    this.enfantMessage = msg;
26  }
```

child.component.ts

src > app > child > child.component.ts > ChildComponent

```
6     styleUrls: ['./child.component.css']
7   })
8   export class ChildComponent {
9     @Input() commentaire: string = '';
10    @Output() messageAuParent: EventEmitter<string> = new EventEmitter<string>();
11
12    EnvoyerMessage() {
13      const message = 'Bonjour, je suis le composant enfant!';
14      this.messageAuParent.emit(message);
15    }
16  }
```

@Output decorator

child.component.html

src > app > child > child.component.html > div

Go to component

```html
<div>
    <h3>Message du parent:</h3>
    <!-- Affichage de la valeur transmise par le parent -->
    <p>{{ commentaire }}</p>

    <!-- Bouton pour envoyer un message au parent -->
  <button (click)="EnvoyerMessage()">Envoyer un message au parent</button>
</div>
```

Message du parent:

Bonjour, je suis le composant parent!

Envoyer un message au parent

Message reçu de l'enfant : Bonjour, je suis le composant enfant!

Cas Pratique

Bonjour, je suis l'enseignant : Mme Lachhab

Question posée à l'étudiant : **Combien de type de classes on peut avoir dans un composant Angular?**

Nom Iyad

3

Envoyer Réponse

La bonne réponse attendue : **3**

Réponse donnée par l'étudiant : **3**

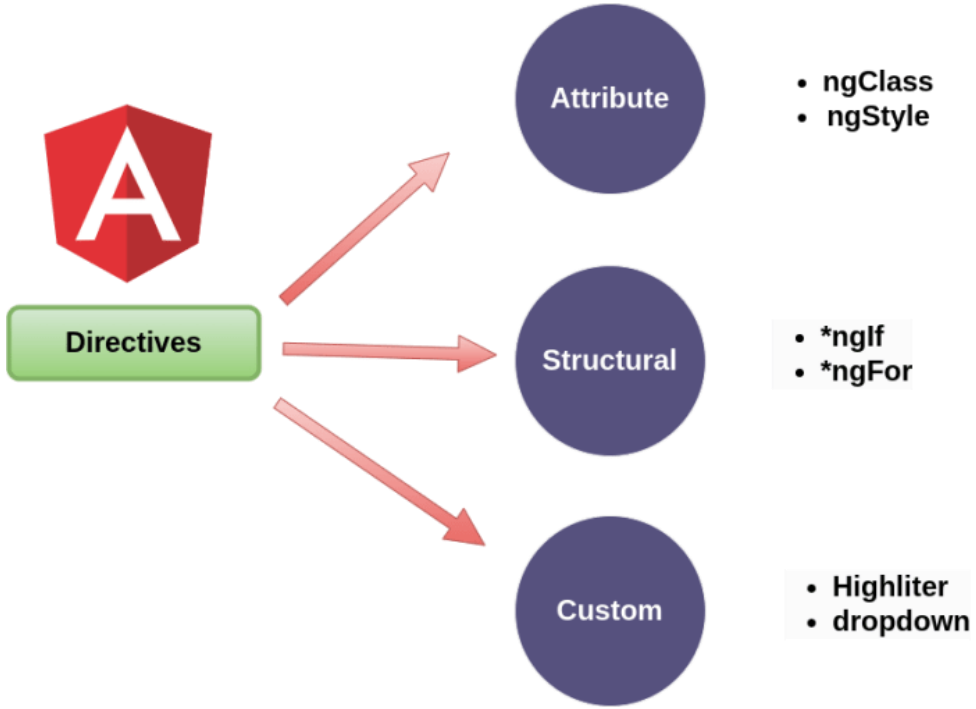
Est-ce une bonne réponse ? **Oui**

Directives

Directives

Une directive est un moyen simple et rapide d'ajouter un élément à différents endroits de l'application.

En TypeScript, une directive est une classe à laquelle on applique le décorateur **@Directive**.



Directives

Les directives **structurelles** : Elles ont pour but de modifier le DOM en ajoutant, enlevant ou replaçant un élément du DOM.

ngIf : *ngIf="<condition>"

ngFor : *ngFor="let <value> of <collection>"

ngSwitchCase : *ngSwitchCase=" value "

Directives

- ngClass
- ngStyle

Les directives de **templating** : Elles ont pour but de modifier **l'apparence** ou le **comportement** d'un élément

ngStyle : **<div [ngStyle]="{color:'red'}">Text **

ngClass :

```
<div [ngClass]="['style1', 'style2']">array of classes
<div [ngClass]="'style1 style2'">string of classes
<div [ngClass]="{'style1': true, 'style2': true}">object of classes</div>
```

Directives

- ngClass
- ngStyle

Les directives de **templating** : Elles ont pour but de modifier **l'apparence** ou le **comportement** d'un élément

ngStyle : **<div [ngStyle]="{color:'red'}">Text **

ngClass :

```
<div [ngClass]="['style1', 'style2']">array of classes
<div [ngClass]="'style1 style2'">string of classes
<div [ngClass]="{'style1': true, 'style2': true}">object of classes</div>
```

Directives : ngIf

1) ngIf Condition

src/app/app.component.ts

```
import { Component } from '@angular/core';
@Component({
selector: 'my-app',
templateUrl: './app.component.html',
styleUrls: [ './app.component.css' ]
})
export class AppComponent {
isShow: boolean = false;
}
```

src/app/app.component.html

```
<h1>Example – ngif </h1>
<div *ngIf="isShow">
<p>
Display if true.
</p>
</div>
```

Directives : ngIf Else

2) ngIf Else Condition

src/app/app.component.ts

```
import { Component } from '@angular/core';
@Component({
selector: 'my-app',
templateUrl: './app.component.html',
styleUrls: [ './app.component.css' ]
})
export class AppComponent {
isShow: boolean = false;
}
```

src/app/app.component.html

```
<h1>Example - </h1>
<p *ngIf="isShow; else ifNotShow<p>
Display if true.
</p>
</div>
<ng-template #ifNotShow>
<p>
Display if not true.
</p>
</ng-template>
```

Directives : ngFor

src/app/app.component.ts

```
import { Component } from '@angular/core';
@Component({
selector: 'my-app',
templateUrl: './app.component.html',
styleUrls: [ './app.component.css' ]
})
export class AppComponent {
students = [
{id: 1, name: ‘ali', email: ‘ali@gmail.com'},
{id: 2, name: ‘sourour', email: ‘sou@gmail.com'},
{id: 3, name: ‘amine', email: ‘amine@gmail.com'},
]
}
```

src/app/app.component.html

```
<table>
<tr>
<th>ID</th>
<th>Name</th>
<th>Email</th>
</tr>
<tr *ngFor="let stu of students">
<td>{{ stu.id }}</td>
<td>{{ stu.name }}</td>
<td>{{ stu.email }}</td>
</tr>
</table>
```

Directives : ngFor

for...in

```
const hotels = ['Douala', 'Lagos', 'Youpougon'];

for (let hotel in hotels) {
  console.log(hotel);
}

// Résultat = 0, 1, 2
```

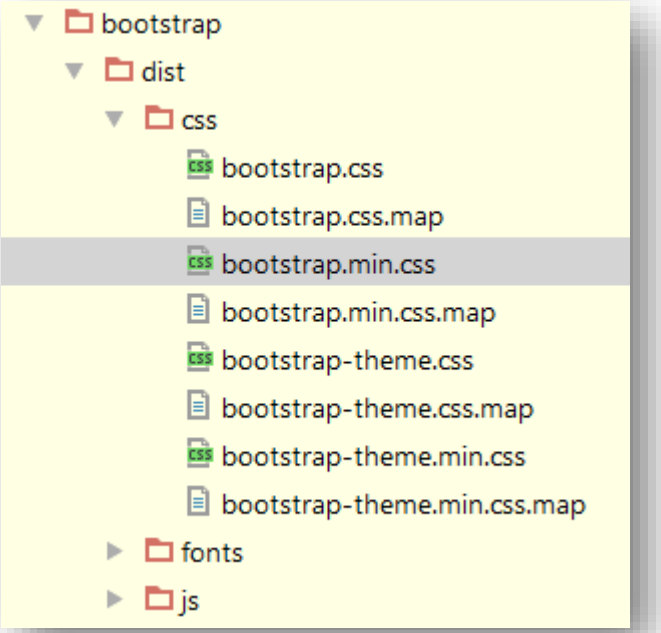
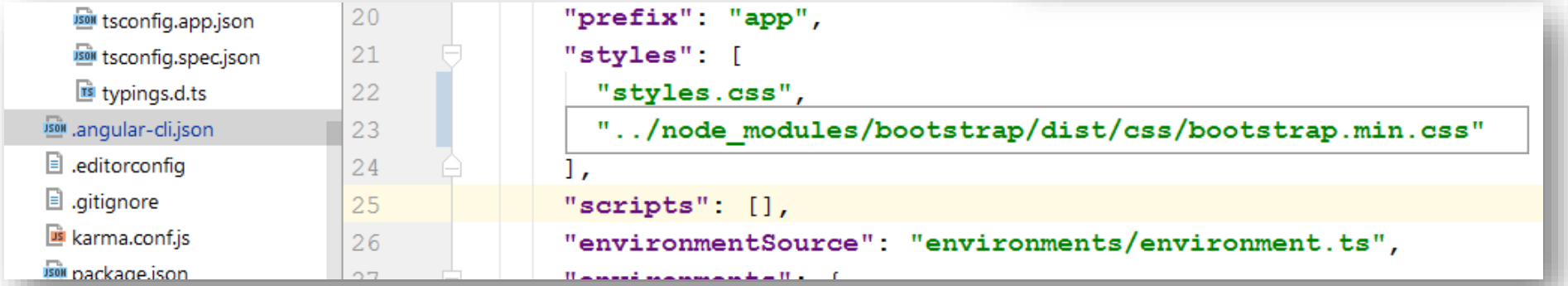
for...of

```
const hotels = ['Douala', 'Lagos', 'Youpougon'];

for (let hotel of hotels) {
  console.log(hotel);
}

// Résultat = Douala, Lagos, Youpougon
```

Directives : ngSwitch

```
export class AppComponent {
  // Variable utilisée pour contrôler le ngSwitch
  choixActuel: string = 'Option1';

  // Méthode pour changer le choix
  changerChoix(choix: string) {
    this.choixActuel = choix;
  }
}
```

```
<div [ngSwitch]="choixActuel">
  <div *ngSwitchCase="'Option1'">Option 1 : Bienvenue dans la section 1.</div>
  <div *ngSwitchCase="'Option2'">Option 2 : Voici la section dédiée à l'option 2.</div>
  <div *ngSwitchCase="'Option3'">Option 3 : Vous êtes dans la section 3.</div>
  <div *ngSwitchDefault>Option inconnue : Merci de sélectionner une option valide.</div>
</div>
```

Ajouter le Framework bootstrap css au projet

- L'un des moyens les plus élégant pour ajouter de nouveau liens vers des fichiers css à index.html est de le faire à l'aide Angular CLI.
- Il faut d'abord installer Bootstrap avec npm:
  - npm install --save bootstrap@3
- Ensuite, ajouter le chemin de bottstrap.min.css dans le fichier .angular-cli.json
- Ensuite démarre le serveur avec **ng serve**